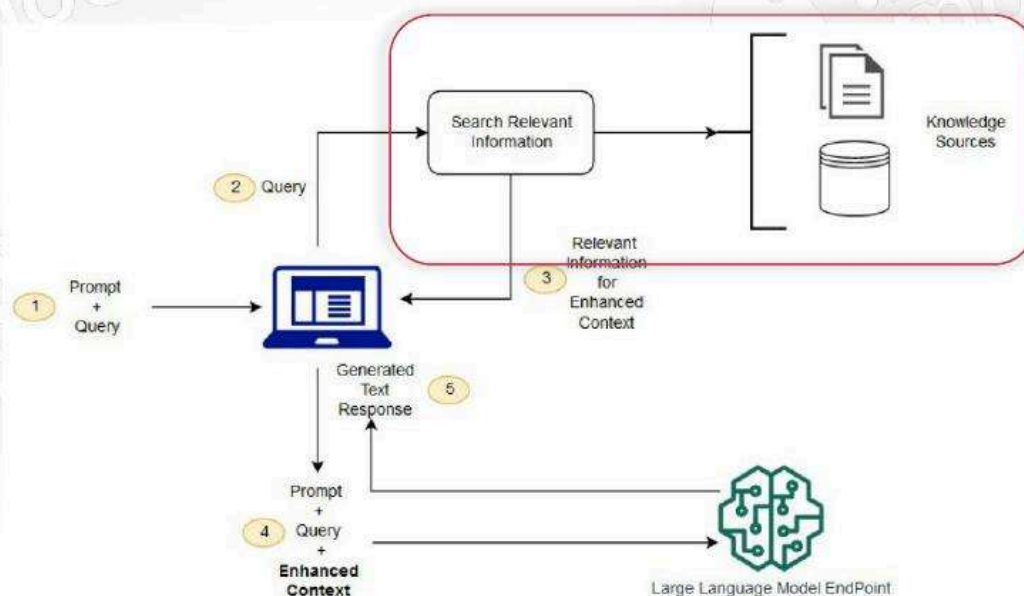


RAG核心三

企业级的向量数据库选型和高效使用

本章简介

- ◆ 企业文档在经过embedding模型进行语义向量化之后，如何进行存储和高效的检索这些知识向量是RAG系统的关键技术点



本章简介

◆ 了解主流向量数据库

◆ 企业级向量数据库的特点

◆ 向量数据库相似性度量

◆ 探索向量数据索引优化技术

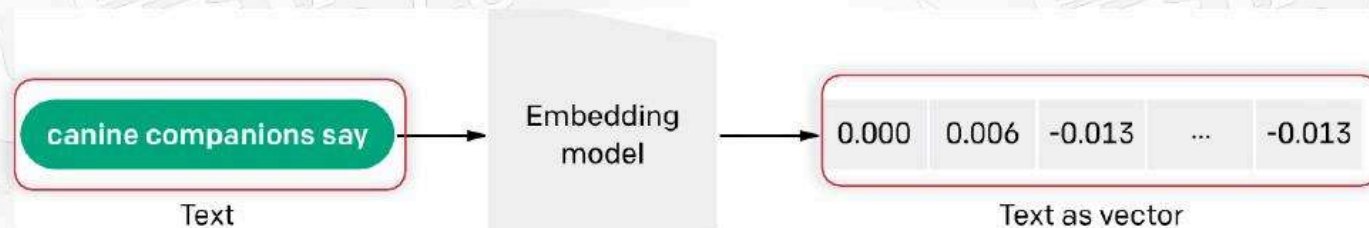
◆ 实战



◆ 总结展望：企业级应用的高可用性设计

了解主流向量数据库

◆ 什么是向量数据库



提供特定的存储结构和索引算法
能够高效地存储、查询和处理向量数据

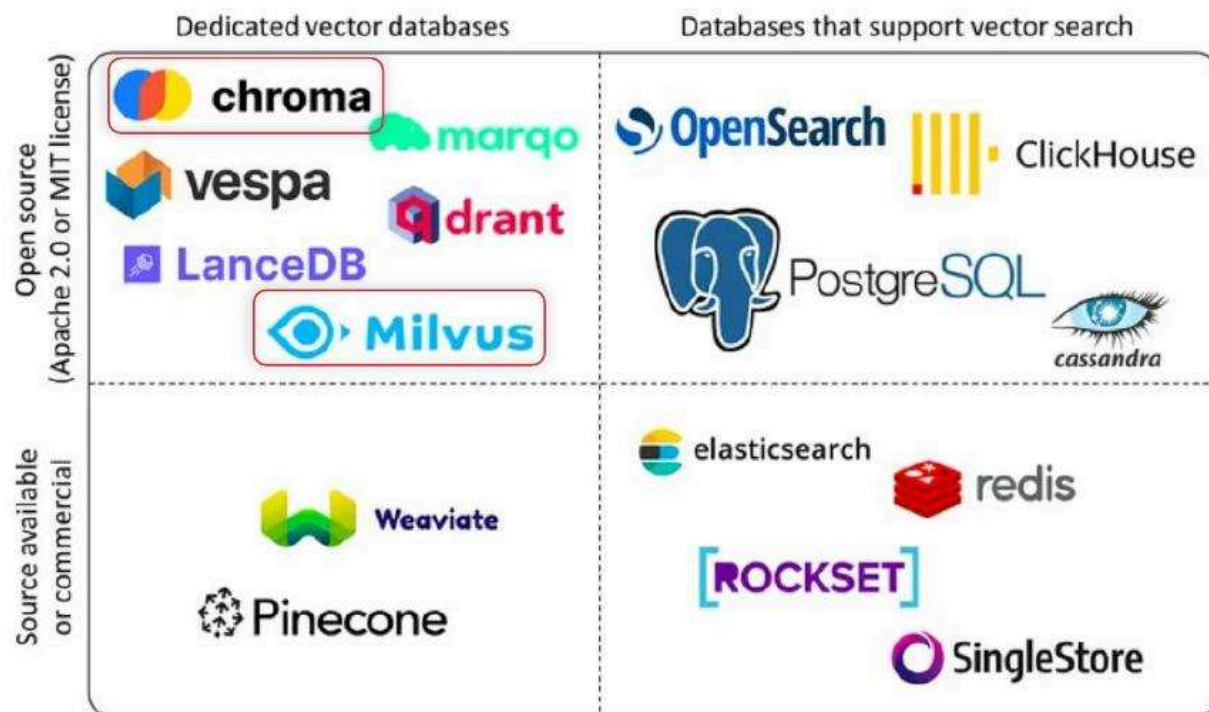
了解主流向量数据库

◆ 向量数据库的功能特点

	关系型数据库	向量数据库
持久化	支持本地存储	支持本地存储
增删改查	需要	需要（侧重在查）
相似度计算	无需	需要
数据存储	表/字段	集合/文档+向量
索引	需要（字段索引，B+数据）	需要（向量索引，IVF，PQ等）
数据类型	多样	字符和数值

了解主流向量数据库

◆ 常用的向量数据库

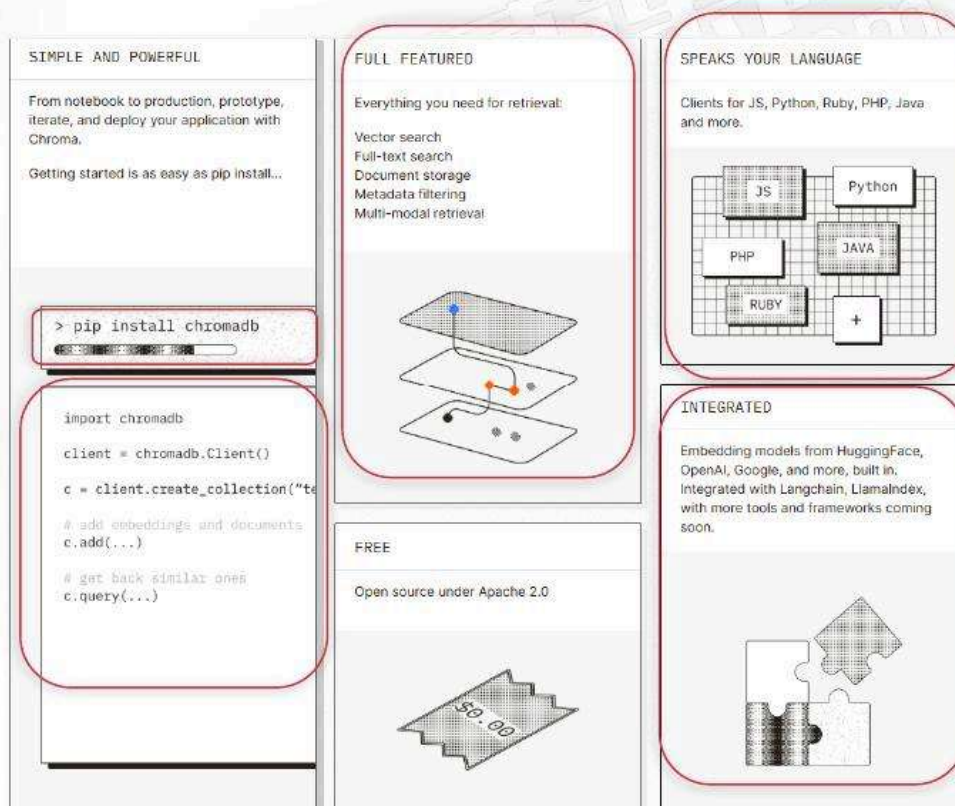


了解主流向量数据库

◆ 常用的向量数据库-chroma



- Chroma 是一个十分易用AI原生的开源向量数据库
- 支持服务端和本地部署
- 支持增删改和各种检索
- 支持python/java/js等多语言调用
- 集成langchain/llamaindex等框架

A graphic summarizing the features of Chroma, organized into six panels. The first panel, 'SIMPLE AND POWERFUL', describes its ease of use from notebook to production and shows a terminal command to install chromadb. The second, 'FULL FEATURED', lists capabilities like vector search and document storage, accompanied by a diagram of a 3D vector space. The third, 'SPEAKS YOUR LANGUAGE', shows client support for various programming languages. The fourth, 'INTEGRATED', highlights compatibility with popular AI frameworks. The fifth, 'FREE', states its open-source status under the Apache 2.0 license. The sixth panel shows a diagram of puzzle pieces, symbolizing integration.

SIMPLE AND POWERFUL

From notebook to production, prototype, iterate, and deploy your application with Chroma.

Getting started is as easy as pip install...

```
> pip install chromadb
```

```
import chromadb
client = chromadb.Client()
c = client.create_collection("test")
# add embeddings and documents
c.add(...)
# get back similar ones
c.query(...)
```

FULL FEATURED

Everything you need for retrieval:

- Vector search
- Full-text search
- Document storage
- Metadata filtering
- Multi-modal retrieval

SPEAKS YOUR LANGUAGE

Clients for JS, Python, Ruby, PHP, Java and more.

INTEGRATED

Embedding models from HuggingFace, OpenAI, Google, and more, built in. Integrated with Langchain, Llamaindex, with more tools and frameworks coming soon.

FREE

Open source under Apache 2.0

了解主流向量数据库

◆ 常用的向量数据库-chroma

建立连接



```
import chromadb
client = chromadb.Client()
# client = chromadb.PersistentClient(path="/path/to/save/to")
```

创建向量集合

```
from chromadb.utils import embedding_functions
model_path = './data/llm_app/embedding_models/gte-large-zh'
em_fn = embedding_functions.SentenceTransformerEmbeddingFunction(model_name=model_path)

client.delete_collection(name="my_collection")
collection = client.create_collection(name="my_collection", embedding_function=em_fn)
```

了解主流向量数据库

◆ 常用的向量数据库-chroma

添加向量到集合中

```
collection.add(  
    documents=["在向量搜索领域，我们拥有多种索引方法和向量处理技术，它们使我们能够在召回率、响应时间和内存使用之间做出权衡。",  
              "虽然单独使用特定技术如倒排文件（IVF）、乘积量化（PQ）或分层导航小世界（HNSW）通常能够带来满意的结果",  
              "GraphRAG 本质上就是 RAG，只不过与一般 RAG 相比，其检索路径上多了一个知识图谱"],  
    metadatas=[{"chapter": 3, "verse": 16}, {"chapter": 4, "verse": 5}, {"chapter": 12, "verse": 5}],  
    ids=["id1", "id2", "id3"]  
)
```

了解主流向量数据库

◆ 常用的向量数据库-chroma

```
get_collection = client.get_collection(name="my_collection", embedding_function=em_fn)
result_by_id = get_collection.get(ids=["id1"], include=["documents", "embeddings", "metadatas"])
```

```
collection.query(
    query_texts=["索引技术有哪些? "],
    n_results=1
)
```

查询

```
collection.query(
    query_texts=["索引技术有哪些? "],
    n_results=2,
    where={"verse": 5},
)
```

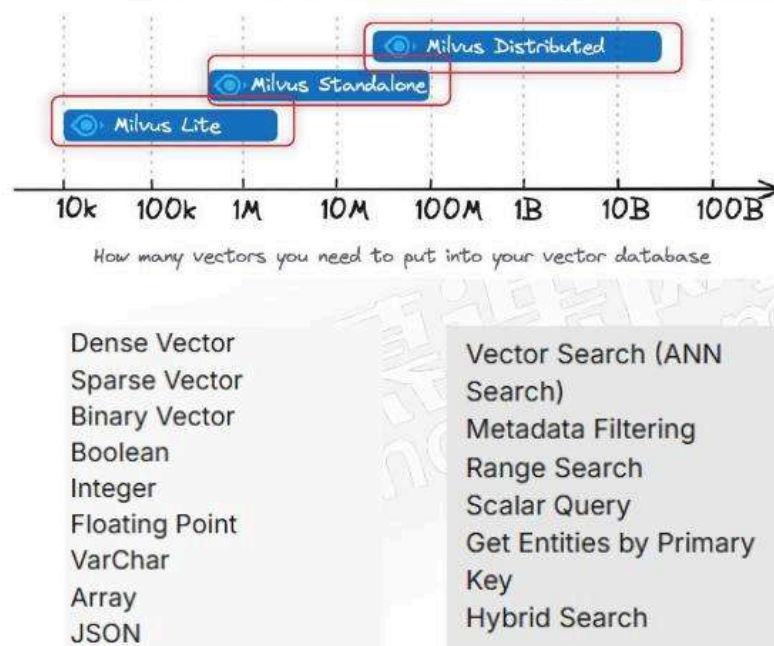
带元数据过滤的查询

了解主流向量数据库

◆ 常用的向量数据库-milvus



- Milvus 是一个高度灵活、可靠且极速的云原生 开源向量数据库
- 支持：本地/服务端/分布式集群部署，可扩展性强
- 支持增删改和各种检索（支持GPU加速）
- 支持python/java/js等多语言调用
- 集成langchain/llamaindex等框架
- 支持访问权限控制



了解主流向量数据库

◆ 常用的向量数据库-milvus

```
print(fmt.format("start connecting to Milvus"))
connections.connect("default", host="127.0.0.1", port="19530")

has = utility.has_collection("hello_milvus")
print(f"Does collection hello_milvus exist in Milvus: {has}")
```

↑ 建立连接

了解主流向量数据库

◆ 常用的向量数据库-milvus

```
fields = [  
    FieldSchema(name="pk", dtype=DataType.VARCHAR, is_primary=True, auto_id=False, max_length=100),  
    FieldSchema(name="random", dtype=DataType.DOUBLE),  
    FieldSchema(name="embeddings", dtype=DataType.FLOAT_VECTOR, dim=dim)  
]  
  
schema = CollectionSchema(fields, "hello_milvus is the simplest demo to introduce the APIs")  
  
print(fmt.format("Create collection `hello_milvus`"))  
hello_milvus = Collection("hello_milvus", schema, consistency_level="Strong")
```

申明字段

建立集合

了解主流向量数据库

◆ 常用的向量数据库-milvus

```
print(fmt.format("Start inserting entities"))
rng = np.random.default_rng(seed=19530)

entities = [
    # provide the pk field because `auto_id` is set to False
    [str(i) for i in range(num_entities)],
    rng.random(num_entities).tolist(), # field random, only supports List
    rng.random((num_entities, dim)),  # field embeddings, supports numpy.ndarray and List
]

insert_result = hello_milvus.insert(entities)

hello_milvus.flush()
print(f"Number of entities in Milvus: {hello_milvus.num_entities}") # check the num_entities
```

准备数据

插入数据

了解主流向量数据库

◆ 常用的向量数据库-milvus

```
print(fmt.format("Start Creating index IVF_FLAT"))  
index = {  
    "index_type": "IVF_FLAT",  
    "metric_type": "L2",  
    "params": {"nlist": 128},  
}  
  
hello_milvus.create_index("embeddings", index)
```

构建索引

了解主流向量数据库

◆ 常用的向量数据库-milvus

混合查询（元数据过滤）

```
print(fmt.format("Start hybrid searching with `random > 0.5`"))
```

```
start_time = time.time()
result = hello_milvus.search(vectors_to_search, "embeddings", search_params, limit=3, expr="random > 0.5", output_fields=["random"])
end_time = time.time()
```

```
for hits in result:
    for hit in hits:
        print(f"hit: {hit}, random field: {hit.entity.get('random')}")
print(search_latency_fmt.format(end_time - start_time))
```

了解主流向量数据库

◆ 常用的向量数据库-chroma vs milvus

简单易用
，适合中
小型项目

	chroma	milvus
部署	本地/服务端/云部署	本地/服务端/云部署（云原生）
扩展性	支持分布式	支持分布式（K8s）
数据管理	增删改查/备份/恢复	增删改查/备份/恢复
向量相似度	Cosine, Euclidean, Dot Product	Euclidean, Cosine, IP, L2, Hamming, Jaccard, Tanimoto
索引	HNSW	HNSW, IVF_FLAT, IVF_SQ8, IVF_PQ, RNSG, ANNOY等
服务管理	无	支持访问控制、监控和日志

功能完备，适合
中大型项目

企业级向量数据库的要求

◆ 向量数据库企业要求

可扩展性



当数据量增加时，是否能够通过增加节点来扩展系统的容量和计算能力

吞吐量



当请求量和数据量增加时，系统能否在短时间内处理更多的请求（QPS）

稳定性



当系统故障时，数据是否丢失，是否能备份和恢复，是否有监控和日志

向量数据库相似性度量

◆ 相似性度量：如何计算两个向量是否相似

距离度量 -----> Euclidean Distance (欧式距离) $d = \sum (A_i - B_i)^2$

越小越相似

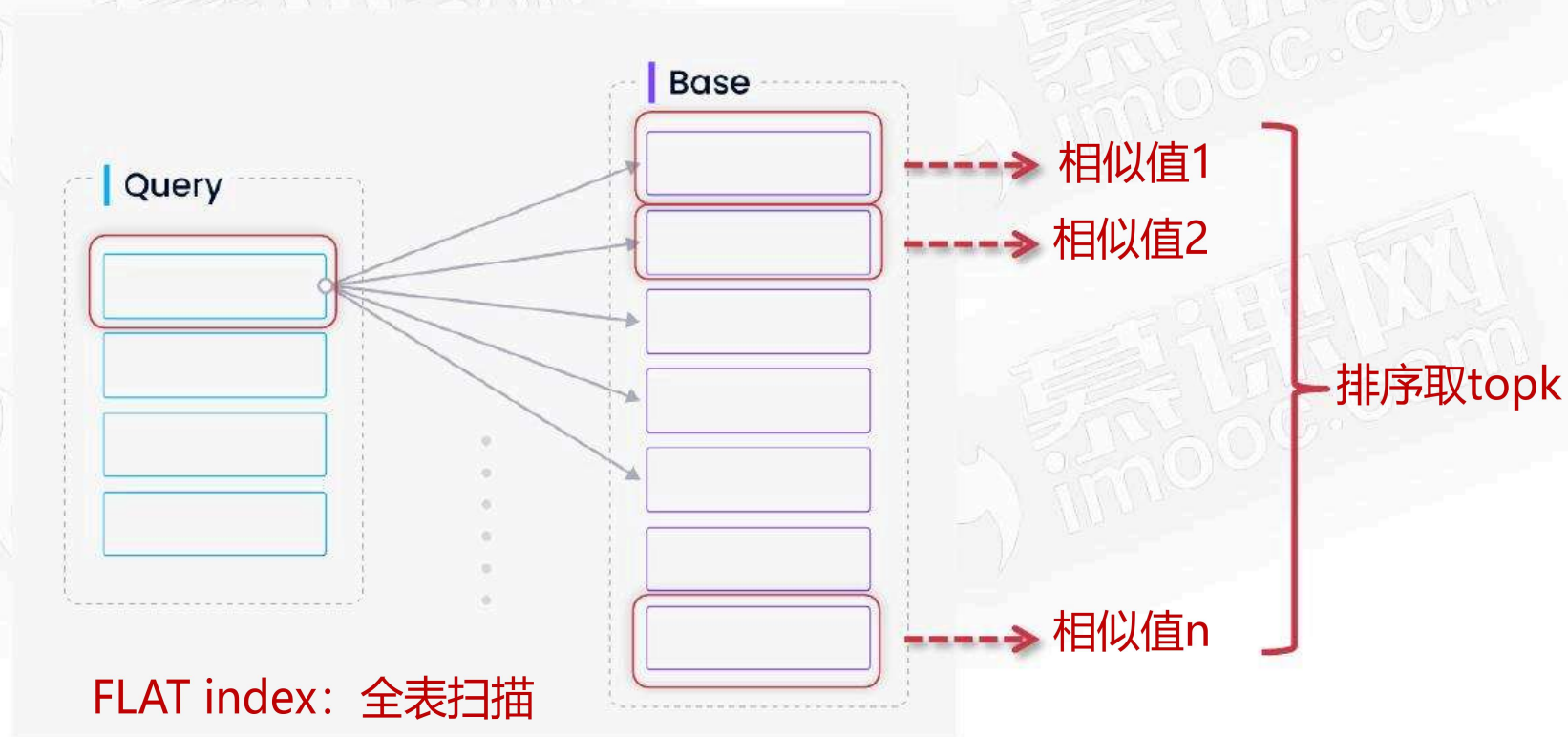
Cosine相似度 ----->

$$\text{cosine similarity} = \cos(\theta) = \frac{\vec{A} \cdot \vec{B}}{\|\vec{A}\| \|\vec{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

越大越相似，A=B时cosine similarity=1

探索向量数据索引优化技术

- ◆ 什么是索引 (index)：用于快速检索和定位数据的方法



探索向量数据索引优化技术

◆ Inverted file index (IVF) : 基于划分的方法

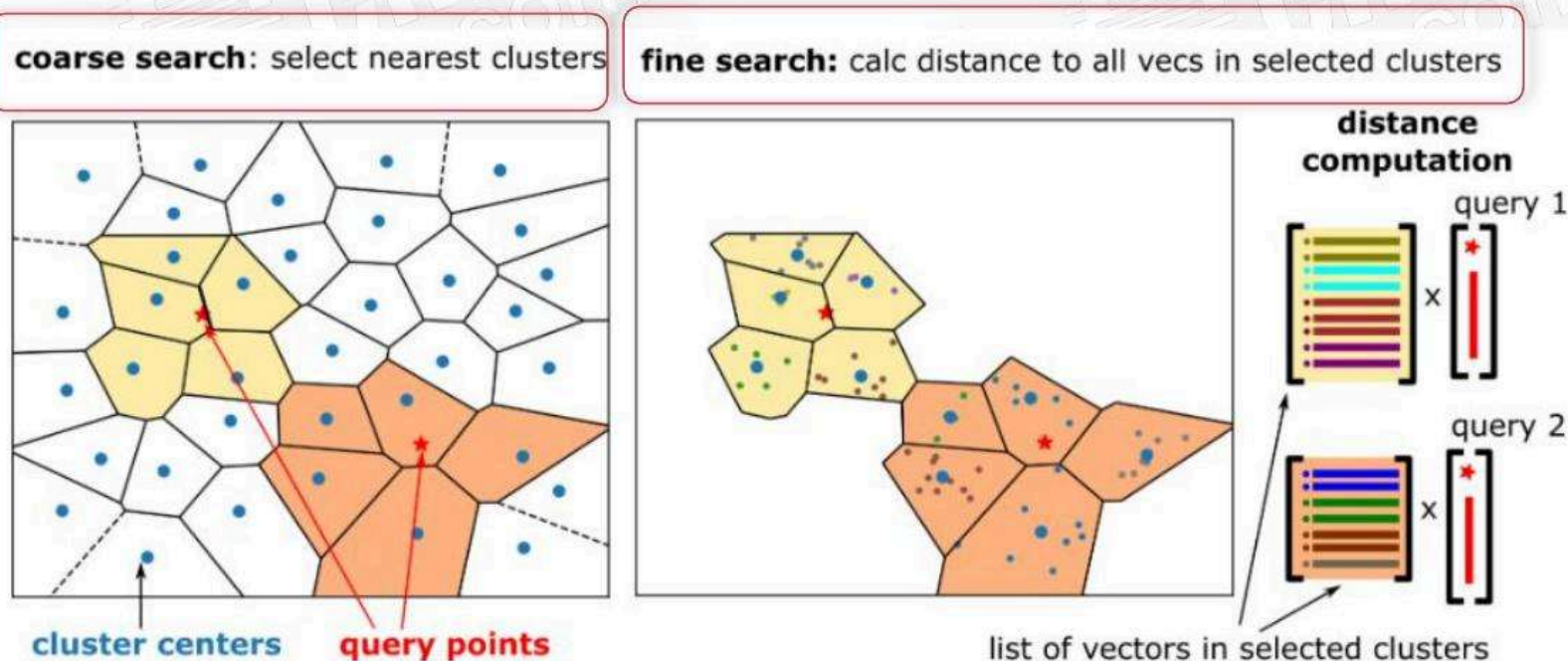
ANN

近似最近邻搜索



探索向量数据索引优化技术

◆ Inverted file index (IVF) : 基于划分的方法



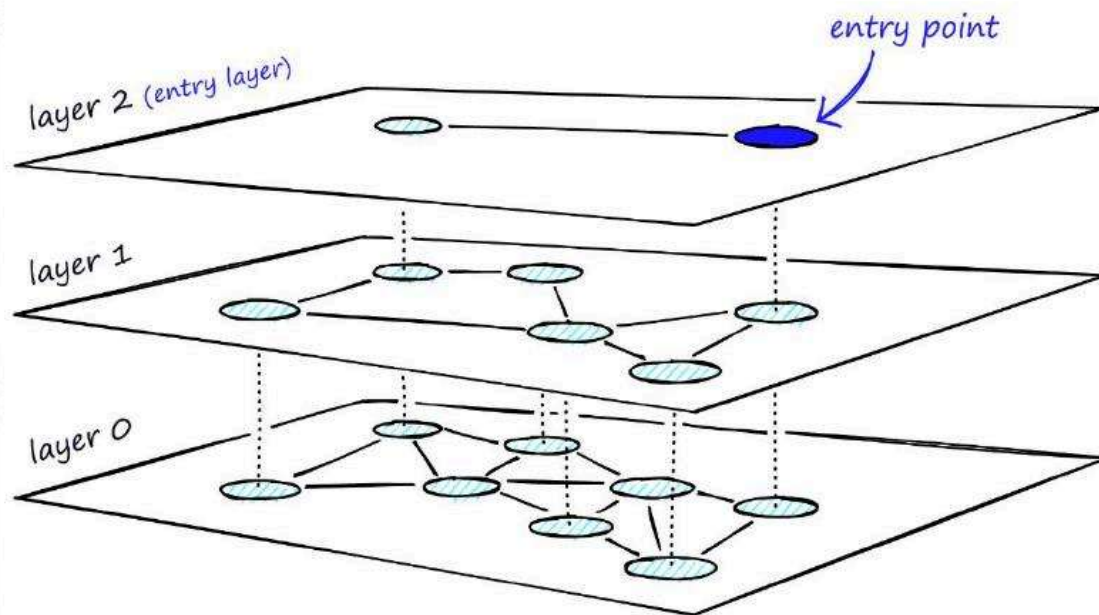
step1.粗筛：找到最近的块中心

step2.细选：在找到的块内做全表扫描

探索向量数据索引优化技术

◆ HNSW: 基于分层的图索引

分层可导航小世界
Hierarchical
Navigable
Small
Worlds



从上往下搜索的粒度从粗到细
最高层有更长的边（用于快速搜索）和较低层有更短的边（用于准确搜索）

1. 分层: 如3层

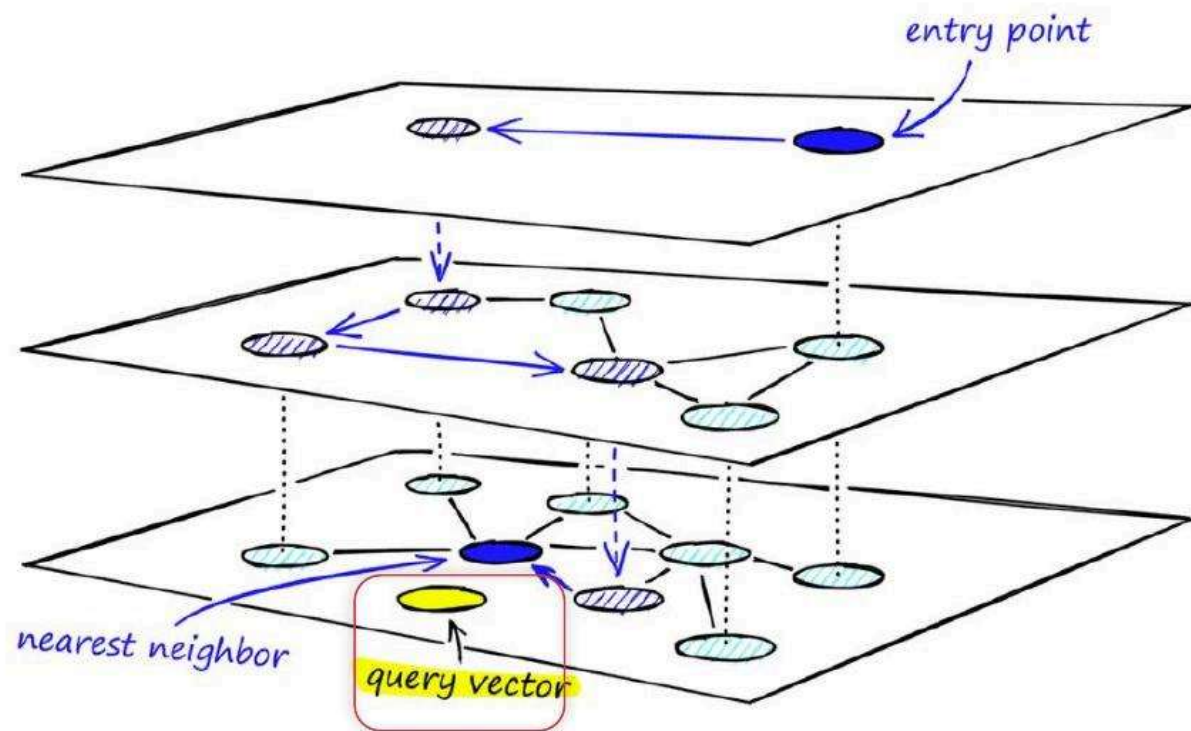
2. 确定数量: 确定每一层的数量（符合指数衰减分布），如10000个向量，第0层有10000个数据元素，第1层有280个元素，第2层只有7个元素

3. 划分数据: layer0是全部数据，从中随机抽取数据到layer1，以此类推，从layer1随机抽取数据到layer2

4. 建立图结构: 每一层中每个数据找到最近k个邻居，构建k近邻图

探索向量数据索引优化技术

◆ HNSW: 基于分层的图索引



1. 从上往下: 搜索开始于最高层

2. 随机切入点: 在layer2: 随机选择一个数据作为切入点entry point, 找到entry point的k个近邻中与查询点最近点, 并以此为新的entry point, 以此类推, 直到entry point不变

3. 上一层的切入点: 在下一层layer1中, 以上一层的entry point为起点, 继续在k邻居里找新的entry point; 以此类推直到在layer0找到最近邻

探索向量数据索引优化技术

◆ 乘积量化 (Product Quantization, PQ) : 量化压缩

高维向量



内存占用大 (数据和索引)

PQ-乘积量化

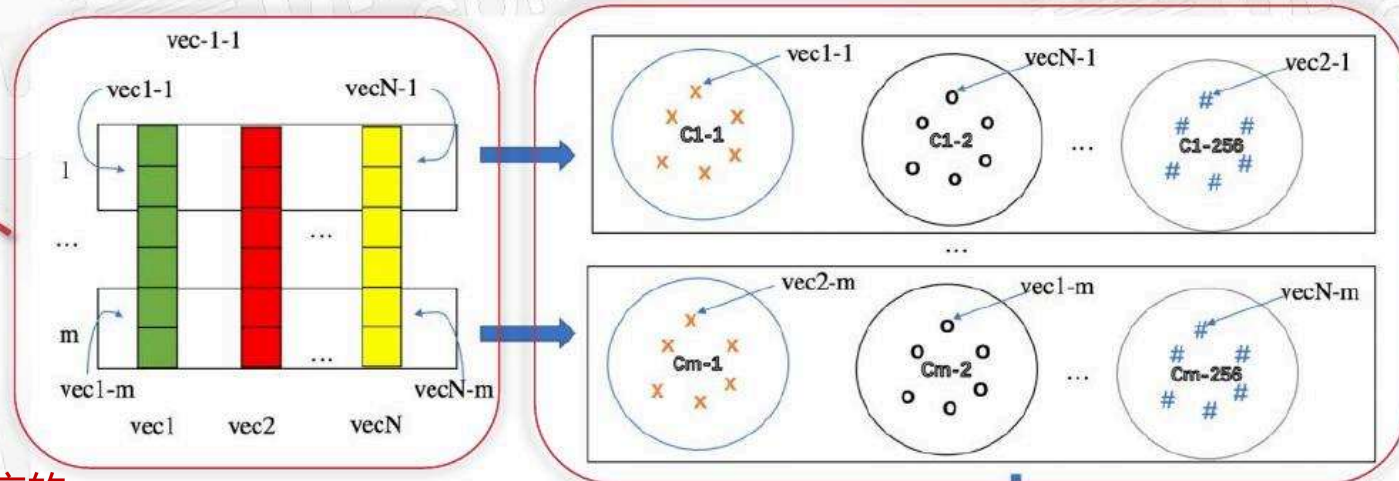


用整型int8代替float来构建索引, 显著压缩高维向量, 实现高达97%的内存节省, 最近邻搜索的速度提高5.5倍

探索向量数据索引优化技术

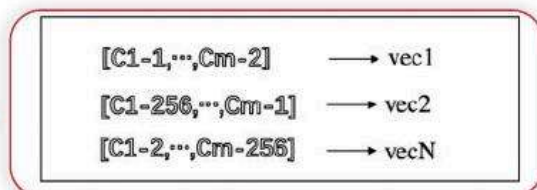
◆ 乘积量化 (Product Quantization, PQ) : 量化压缩

向量切分为m个子向量



对所有子向量进行聚类得到256个块

用子向量对应的块ID来表示原始向量，并以此为构建索引



维度降为m维，取值为：0-255

探索向量数据索引优化技术

◆ 乘积量化 (Product Quantization, PQ) : 量化压缩

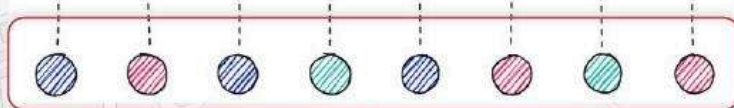
Query向量



切分m个子向量



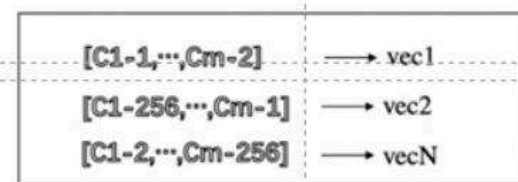
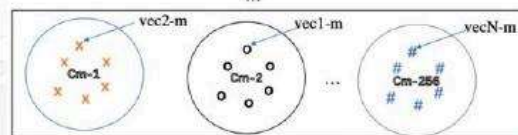
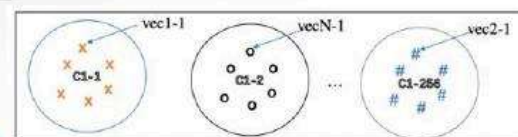
计算获取每个子向量对应的聚类块中心ID



用聚类块中心ID表示Query向量

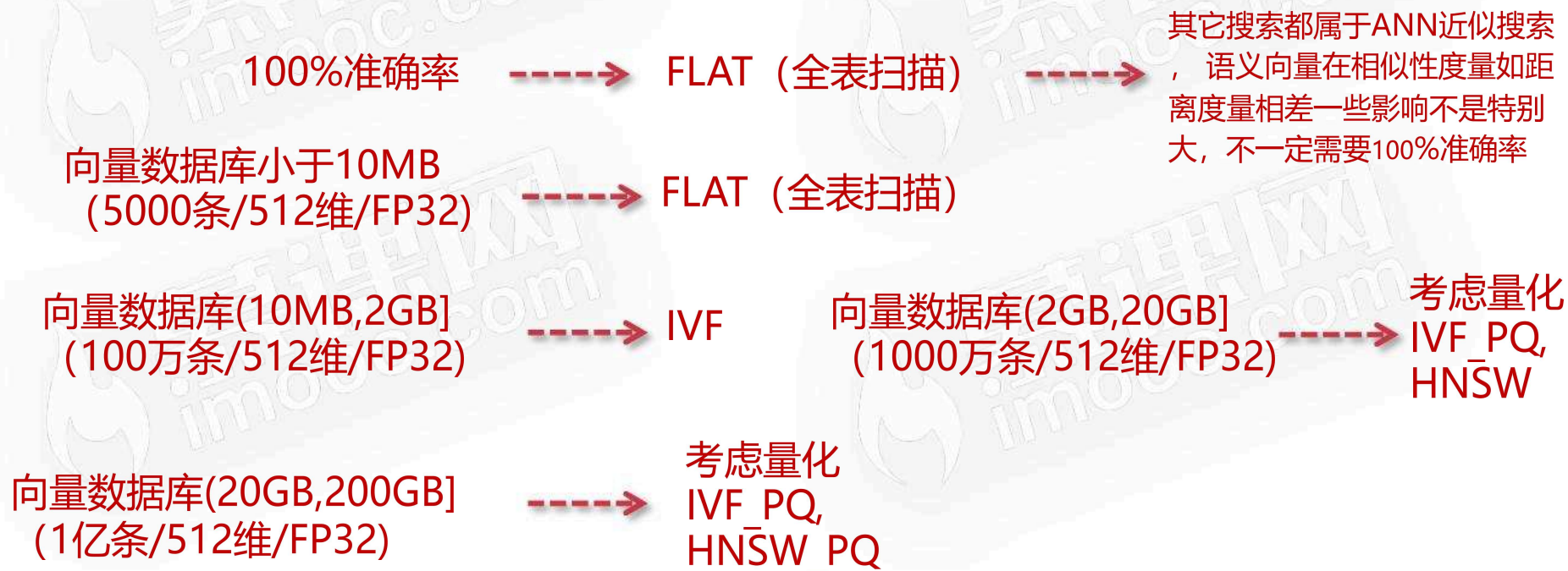


在索引中寻找最近的原始向量



探索向量数据索引优化技术

◆ 选择索引建议：准确率和数据量



总结与展望

◆ 总结

向量数据库



作用：存储高维的embedding向量

特点：提供相似度搜索，IVF/PQ/HNSW索引提供高性能近似搜索

工具：常用开源数据库：chroma/milvus

总结与展望

◆ 总结

企业向量数据库



要求：可扩展/高吞吐/稳定性

chroma：比较适合千万以内数据场景

milvus：云原生，分布式，可扩展性好，支持亿级数据毫秒响应，支持权限控制

总结与展望

◆ 企业级应用的高可用性设计

居安思危，墨菲定律



系统故障了怎么办？

高可用目标



可靠性、可用性和可维护性，确保系统在各种情况下都能保持运行

设计原则



分布式集群部署：负载均衡（nginx），避免单点故障

容错和冗余设计：主备集群，自动切换

监控和告警：便于即时恢复